

使用 BigQuery 和 Gemini 模型建構 AI 輔助車輛市集

來源：<https://codelabs.developers.google.com/next26/bigquery-ai-datascience?hl=zh-tw>

步驟數：11

使用 BigQuery ML、多模態版 Gemini 視覺、語意搜尋和 Next.js，建構智慧車輛市集。

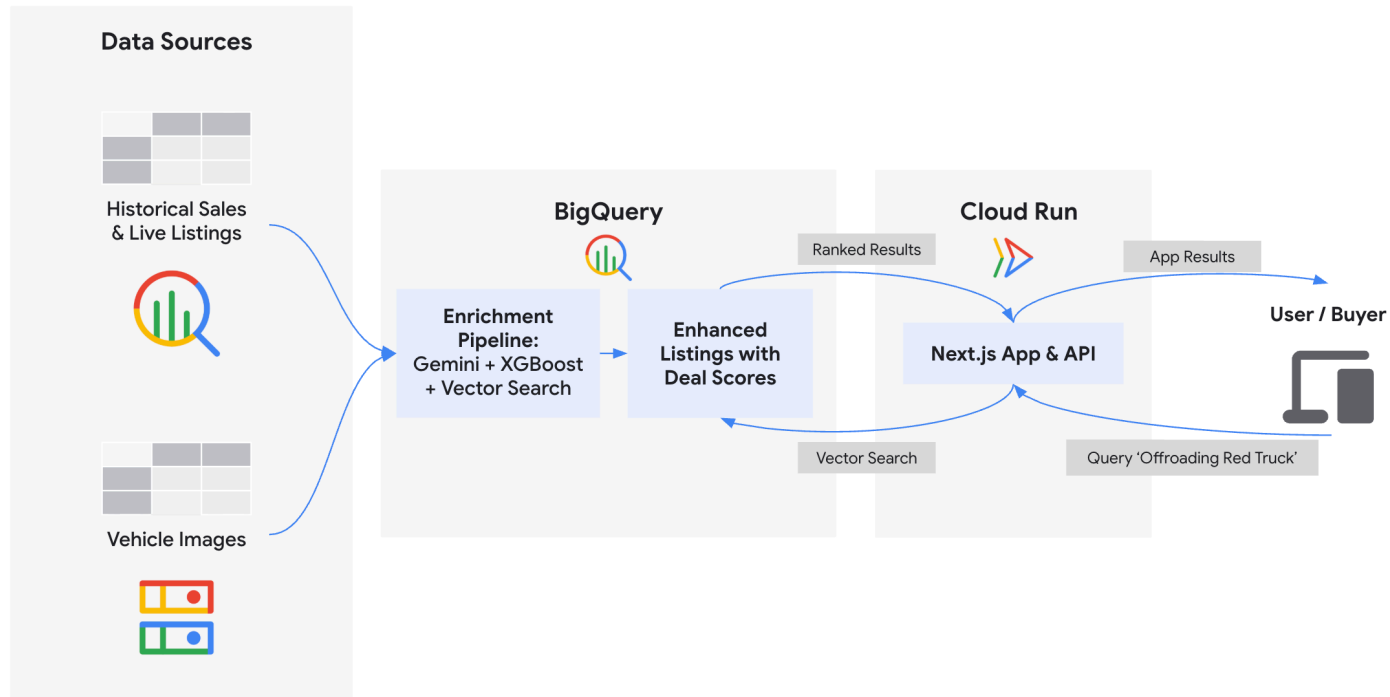
目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 取得程式碼和設定資料](#)
4. [4. 多模態視覺化擷取](#)
5. [5. 使用 XGBoost 進行預測性定價](#)
6. [6. 語意嵌入和真實性偵測](#)
7. [7. 生成式交易評分](#)
8. [8. 部署前端應用程式](#)
9. [9. 探索 Cymbal Autos 應用程式](#)
10. [10. 清除所用資源](#)
11. [11. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將建構後端，並部署「Cymbal Autos」這個線上車輛市集的前端。您將使用 [BigQuery](#) 和 [Gemini Enterprise Agent Platform](#) 上的 [Gemini 模型](#) 檢查車輛相片、使用 [BigQuery ML](#) 預測價格、使用向量嵌入偵測詐騙資訊，以及計算綜合交易分數。最後，您會在部署至 [Cloud Run](#) 的 Next.js 前端顯示這些洞察資料。



學習內容

- 使用 [ObjectRef](#) 將 BigQuery 連結至非結構化 Cloud Storage 圖片
- 使用 [BigQuery](#) 和 [Gemini 模型](#) 從相片中擷取車輛屬性
- 使用 [BigQuery ML](#) 訓練 XGBoost 迴歸模型，預測公允市場價格
- 嵌入車輛說明並執行 [VECTOR_SEARCH](#)，找出潛在詐騙和值得信賴的車輛資訊
- 計算每項產品資訊的全面性特惠分數，同時使用 [AI.SCORE](#) 納入賣家說明中的條件信號
- 匯出資料，並將 Next.js Marketplace 應用程式部署至 Google Cloud Run

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 雲端專案
- 對 SQL、Python 和 Google Cloud 有基本的瞭解
- 具備足夠的 IAM 權限，可啟用 API、建立資源及指派權限 (例如專案擁有者)

本程式碼研究室適合中階開發人員。

本程式碼研究室建立的資源費用應低於 \$5 美元。

步驟 2 · 約 5 分鐘

2. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

您將使用 [Google Cloud Shell](#) 下載程式碼、執行設定指令碼，以及部署應用程式。

1. 按一下 [Google Cloud 控制台](#) 頂端的「啟用 Cloud Shell」。



2. 連線至 Cloud Shell 後，請驗證工作階段，確保應用程式可以存取 Google Cloud API。按照提示授權 Cloud Shell：

```
gcloud auth application-default login
```

3. 設定 Google Cloud 專案 ID，以及 Google Cloud Storage bucket 的不重複名稱 (用於儲存原始資料)：

```
export PROJECT_ID=$(gcloud config get-value project)
export USER_BUCKET="cymbal-autos-${PROJECT_ID}"

gcloud config set project $PROJECT_ID
```

畫面會顯示類似以下的訊息：

```
Your active configuration is: [cloudshell-####]
Updated property [core/project]
```

啟用 API

在 Cloud Shell 中執行下列指令，啟用本程式碼研究室的所有必要 API：

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  artifactregistry.googleapis.com \  
  bigquery.googleapis.com \  
  bigqueryconnection.googleapis.com \  
  cloudbuild.googleapis.com \  
  run.googleapis.com
```

執行成功後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```

3. 取得程式碼和設定資料

首先，請下載示範資產並設定環境變數。

1. 在 Cloud Shell 中複製 `devrel-demos` 存放區，然後前往專案目錄：

```
git clone https://github.com/GoogleCloudPlatform/devrel-demos.git
cd devrel-demos/data-analytics/cymbal-autos-multimodal
```

請務必從這個複製的目錄中執行所有後續的 `gcloud` 和 `bq` 指令。

2. 執行指令碼，將資料複製到您的環境。這個指令碼會將本機存放區資料集同步到您的個人 Cloud Storage bucket，並從公開 bucket 擷取車輛圖片：

```
chmod +x scripts/setup/*.sh
./scripts/setup/00_copy_data.sh
```

之後，您應該會看到類似以下的訊息：

```
Average throughput: 87.8MiB/s
Data copy complete!
```

3. 接著，請設定 BigQuery Cloud 資源連結。如要分析 Cloud Storage 中的非結構化圖片，並直接從 SQL 查詢呼叫 Agent Platform 模型，BigQuery 必須將 IAM 權限委派給基礎服務帳戶。這個指令碼會建立安全連線，並授予必要的 Vertex AI 使用者和服務使用情形消費者角色 (傳播時間約為一分鐘)：

```
./scripts/setup/01_setup_api_connection.sh
```

您應會看見類似以下的訊息：

```
Environment setup complete! Your BigQuery connection is ready.
```

4. 最後，建立初始 BigQuery 資料集並載入原始表格資料。這會建立 `model_dev` 資料集並填入起始資料表，在您編寫任何機器學習查詢之前，先設定基礎：

```
./scripts/setup/02_load_to_bq.sh
```

您應會看見類似以下的訊息：

```
=====
BigQuery load complete!
=====
```

4. 多模態視覺化擷取

在為車輛資訊評分前，您會從數百張原始相片中擷取結構化資料 (例如顏色、車身樣式或外觀損壞)。只要運用 [ObjectRef 函式](#) 和 Agent Platform 中託管的 Gemini 模型，就能生成這些功能，完全不需要移動任何檔案或撰寫複雜的資料管道。這項擷取作業會直接為前端應用程式的「視覺條件」🌟 徽章提供資訊。

1. 在新瀏覽器分頁中開啟 [BigQuery Studio](#)。
2. 按一下「+ 撰寫新查詢」按鈕。在本程式碼研究室中，您會使用 SQL 編輯器與 SQL 程式碼互動。
3. 建構機器學習擷取器之前，可以先快速查看原始圖片。執行下列查詢，即可查看每個房源儲存在 Google Cloud Storage 中的圖片 URI 陣列：

```
SELECT auction_id, item_name, description, images
FROM `model_dev.vehicle_metadata` LIMIT 5;
```

4. 現在，請在 BigQuery Studio SQL 編輯器中貼上下列 SQL，建立含有 `image_ref` 欄的新資料表。按一下「執行」。

```
CREATE OR REPLACE TABLE `model_dev.vehicle_multimodal` AS
SELECT
  *,
  ARRAY(
    SELECT OBJ.FETCH_METADATA(OBJ.MAKE_REF(uri, 'us.conn'))
    FROM UNNEST(images) AS uri
  ) AS image_ref
FROM `model_dev.vehicle_metadata`;
```

什麼是 `OBJ.MAKE_REF`？[OBJ.MAKE_REF](#) 函式可讓您直接從 BigQuery 資料表參照 Cloud Storage 中的物件。BigQuery 會將這視為另一個資料欄，可供操控或用於推論。

5. 查看您剛建立的新 `image_ref` ObjectRef 欄。新資料表現在有 ObjectRef 資料欄，可對圖片本身執行權限。執行下列查詢即可查看：

```
SELECT auction_id, item_name, description, image_ref
FROM `model_dev.vehicle_multimodal` LIMIT 5;
```

6. 現在請使用 [AI.GENERATE](#) 和 [AI.CLASSIFY](#) 分析圖片。`AI.GENERATE` 會提示 Gemini 擷取狀況分數和一句話的損壞摘要，而 `AI.CLASSIFY` 則會嚴格分類車輛的車身樣式和顏色。

SQL 中的提示工程：請注意下方查詢中提供的 `output_schema => 'condition INT64, description_summary STRING'` 字串。這會指示 Gemini 模型傳回結構化 JSON，BigQuery 會將其原生轉換為具型別的資料欄。

執行下列查詢，將這些洞察資料擷取到專屬的特徵資料表中。這項作業大約需要 3 分鐘。


```

CREATE OR REPLACE TABLE `model_dev.vehicle_vision_features` AS
WITH generated_data AS (
  SELECT
    auction_id,
    AI.GENERATE(
      prompt => ('Rate the condition of this car on a scale from 0-100. Output a 1 sentence
description of any glaring red flags', image_ref),
      output_schema => 'condition INT64, description_summary STRING'
    ).* EXCEPT(full_response,status)
  FROM
    `model_dev.vehicle_multimodal`
),

-- Object-centric Classifications
classified_data AS (
  SELECT
    auction_id,
    AI.CLASSIFY(
      ('What type of automobile is this?', image_ref[0]),
      categories => ['Truck', 'Sedan', 'SUV']) AS body_style,
    AI.CLASSIFY(
      ('Color of the exterior of the automobile', image_ref[0]),
      categories => ['Black', 'White', 'Silver', 'Gray', 'Red', 'Blue', 'Brown', 'Green',
'Beige', 'Gold']) AS color,
    AI.CLASSIFY(
      ('Color of the interior of the automobile', image_ref[0]),
      categories => ['Black', 'Gray', 'Beige', 'Tan', 'Brown', 'White', 'Red']) AS interior
  FROM `model_dev.vehicle_multimodal`
)

-- Join the AI insights back together into the final feature table
SELECT
  g.auction_id,
  g.condition,
  g.description_summary,
  c.body_style,
  c.color,
  c.interior
FROM generated_data g
JOIN classified_data c ON g.auction_id = c.auction_id;

```

7. 如要自行查看生成的特徵，請執行下列查詢，或直接查看下方的螢幕截圖：

```

SELECT auction_id, condition, description_summary, body_style, color, interior FROM
`model_dev.vehicle_vision_features` LIMIT 5;

```

Row	auction_id	condition	description_summary	body_style	color	interior
1	355481	15	The silver SUV has sustained severe front-end collision damage, including a buckled hood and detached bumper, which are major red flags	SUV	Silver	Black
2	355302	25	The primary red flag is the severe front-end collision damage on the driver's side of this blue vehicle, which likely	SUV	Blue	Black

本節重點：您直接從 BigQuery 存取原始圖片，並使用 Gemini 模型擷取結構化視覺特徵，不必移動任何檔案。

5. 使用 XGBoost 進行預測性定價

如要判斷車輛是否物超所值，必須先取得公允市價的可靠基準。您不必將資料擷取到本機指令碼或筆記本來訓練模型，而是可以使用標準 SQL，直接在 BigQuery 內訓練 [XGBoost 模型](#)。這項價格預測會驅動前端應用程式的  公允市場價值邏輯。

1. 返回「BigQuery Studio」分頁。
2. 首先，請查看訓練資料集。與可交易車輛物件不同，這個 `synthetic_cars` 資料表包含 10 萬筆歷來銷售資料，可用於訓練模型。執行這項快速查詢來一探究竟：

```
SELECT
  *
FROM
  `model_dev.synthetic_cars`
LIMIT 10;
```

3. 現在，請執行下列 SQL，訓練 XGBoost 迴歸模型。這個模型會從這 10 萬筆歷史記錄中，瞭解里程數、年份、廠牌和外觀狀況等屬性如何影響價格：

```
CREATE OR REPLACE MODEL `model_dev.car_price_model`
OPTIONS(
  MODEL_TYPE = 'BOOSTED_TREE_REGRESSOR',
  INPUT_LABEL_COLS = ['selling_price'],
  MAX_ITERATIONS = 15,
  TREE_METHOD = 'HIST'
) AS
SELECT
  * EXCEPT(vin, sale_date, market_value, seller)
FROM
  `model_dev.synthetic_cars`;
```

喝杯咖啡：在 BigQuery 中訓練這個 XGBoost 模型通常需要約 5 到 7 分鐘才能完成。

4. 如要預測目前進行中的車輛刊登價格，你必須先將所有相關輸入特徵集中在一處。執行這項 SQL，將結構化車輛中繼資料與您剛產生的視覺化擷取特徵合併：

```
CREATE OR REPLACE TABLE `model_dev.vehicle_prediction_features` AS
SELECT
  meta.auction_id,
  meta.model_year,
  meta.make,
  meta.model,
  meta.mileage,
  meta.transmission_type,
  meta.state,
  COALESCE(vision.body_style, 'Unknown') AS body_style,
  COALESCE(vision.condition, 50) AS condition,
  COALESCE(meta.color, vision.color, 'Unknown') AS color,
  COALESCE(vision.interior, 'Unknown') AS interior
FROM `model_dev.vehicle_metadata` meta
LEFT JOIN `model_dev.vehicle_vision_features` vision
  ON meta.auction_id = vision.auction_id;
```

5. 最後，預測每個持續進行的車輛刊登的公允市場價值。執行下列查詢，將匯總特徵饋送至新訓練的模型，並將數值輸出內容儲存至安全的預測資料表：

```
CREATE OR REPLACE TABLE `model_dev.vehicle_price_predictions` AS
SELECT
  auction_id,
  ROUND(predicted_selling_price, 2) AS predicted_market_value
FROM ML.PREDICT(
  MODEL `model_dev.car_price_model`,
  (SELECT * FROM `model_dev.vehicle_prediction_features`)
);
```

什麼是 `ML.PREDICT` ？ `ML.PREDICT` 函式會將訓練完成的模型套用至新資料，以產生預測結果。直接在 SQL 中執行這項函式時，BigQuery 會自動在資料集之間擴充推論運算，避免維護外部推論端點的額外負擔。

6. 現在請驗證模型的輸出內容。執行這項快速查詢，即可預覽即時車輛資訊的預測市場價值：

```
SELECT * FROM `model_dev.vehicle_price_predictions` LIMIT 5;
```

Row	auction_id	predicted_market_value
1	355819	1003.12
2	356415	1158.27
3	355533	1247.59
4	356100	1796.68
5	356302	1905.81

本節重點：您使用 10 萬筆交易樣本訓練 XGBoost 迴歸模型，並執行批次推論，預測資料集中每個有效車輛刊登的公允市場價值。

6. 語意嵌入和真實性偵測

在本節中，您將執行兩個不同的嵌入管道，為車輛市集啟用智慧功能：

1. **多模態圖片搜尋**：將原始車輛相片轉換為向量空間，讓使用者以自然語言搜尋 (例如「可靠的工作用卡車」)。
2. **文字嵌入和相似度搜尋**：將車輛說明文字轉換為向量嵌入，然後使用 `VECTOR_SEARCH` 比較現有刊登資訊與已知的潛在詐騙或車迷個人資料。系統會據此計算買家在應用程式中看到的 🔍 **真實性分數**。

什麼是 `VECTOR_SEARCH` ？ `VECTOR_SEARCH` 函式會計算高維度向量之間的語意距離。直接在 GoogleSQL 查詢中執行近似最鄰近相似度搜尋。

1. 首先，您需要為車輛產品資訊生成多模態嵌入。使用 `gemini-embedding-2-preview` 模型時，您可以將圖片和文字輸入至完全相同的嵌入內容。雖然這個模型完全有能力同時處理多種模態，但在這個特定案例中，我們只會嵌入車輛圖片。這項功能可為前端應用程式的「語意搜尋」列提供支援，讓買家使用自然語言 (例如「可靠的皮卡車」) 快速擷取相符的房源。執行這項查詢，使用 `AI.EMBED` 生成多模態向量：

```
CREATE OR REPLACE TABLE `model_dev.vehicle_images_embedded` AS
SELECT
  auction_id,
  AI.EMBED(
    STRUCT(image_ref),
    endpoint => 'gemini-embedding-2-preview').result AS multimodal_embedding
FROM `model_dev.vehicle_multimodal`
WHERE ARRAY_LENGTH(image_ref) > 0;
```

2. 接著，您會檢查先前載入的風險設定檔資料。請注意，這份清單包含已知的詐騙類型，以及正當的愛好者範例房源資訊。執行這項查詢即可查看基準設定檔：

```
SELECT profile_id, profile_type, description
FROM `model_dev.seller_risk_profiles`;
```

3. 現在，請將這些原始風險說明轉換為向量嵌入。您可以運用專門的文字嵌入模型 (`text-embedding-005`)，嚴格評估剛才預覽的文字。貼上下列 SQL，然後按一下「執行」，即可嵌入基準設定檔：

```
CREATE OR REPLACE TABLE `model_dev.seller_risk_profiles_embedded` AS
SELECT
  profile_id,
  description AS content,
  profile_type,
  AI.EMBED(description, endpoint => 'text-embedding-005').result AS text_embedding
FROM `model_dev.seller_risk_profiles`;
```

4. 接著，為實際的即時車輛商品目錄生成可比較的嵌入內容。執行這項查詢，將每輛車的原始 HTML 說明轉換為向量空間，以便與基準設定檔進行比較：

```
CREATE OR REPLACE TABLE `model_dev.vehicle_descriptions_embedded` AS
SELECT
  auction_id,
  description AS content,
  AI.EMBED(description, endpoint => 'text-embedding-005').result AS text_embedding
FROM `model_dev.vehicle_metadata`
WHERE description IS NOT NULL;
```

5. 最後，執行向量搜尋，計算即時房源與基準設定檔之間的語意距離。執行下列 SQL 進行對應。數學距離越小，表示房源與已知詐騙群組的相似度越高；距離越大，則表示說明內容合法。

```
CREATE OR REPLACE TABLE `model_dev.vehicle_authenticity_scores` AS
SELECT
  scam_search.query.auction_id,
  CAST(
    GREATEST(0.0, LEAST(100.0, ROUND((MIN(scam_search.distance) - 0.33) / 0.12 * 100.0)))
    AS INT64
  ) AS authenticity_score
FROM VECTOR_SEARCH(
  TABLE `model_dev.seller_risk_profiles_embedded`,
  'text_embedding',
  (
    SELECT text_embedding, auction_id
    FROM `model_dev.vehicle_descriptions_embedded`
  ),
  top_k => 15,
  distance_type => 'COSINE'
) AS scam_search
WHERE scam_search.base.profile_type = 'scam'
GROUP BY 1;
```

為什麼要調整數學運算？ 您可能會注意到計算結果 $(\text{MIN}(\text{scam_search.distance}) - 0.33) / 0.12 * 100.0$ 。由於所有房源描述的項目（車輛）在實體上非常相似，因此基本餘弦距離自然會介於 0.33 和 0.45 之間。查詢會以數學方式將微小的 0.12 變異數延展為乾淨的 0 到 100 `authenticity_score`，前端應用程式處理起來會容易許多。

這個表格的內容可能如下所示：

Row	auction_id	authenticity_score
1	356015	74
2	356454	65
3	356163	69
4	355459	83
5	356271	80

本節回顧：您已為前端搜尋列生成多模態嵌入內容，並直接在 BigQuery 中使用向量搜尋，根據已知的詐騙設定檔評估原始 HTML 文字資訊。

7. 生成式交易評分

您現在已透過多種不同的機器學習技術，產生結構化資料集，所有作業完全在 BigQuery 中完成：視覺化擷取、XGBoost 模型 (用於預測公允市場價值) 和向量搜尋嵌入。

最後一個步驟是將這些 AI 信號合併為整合式檢視畫面，做為前端應用程式的最終**交易分數**。

1. 首先，請將原始中繼資料與 AI 擷取的視覺特徵、預測價格輸出內容和語意真實性分數合併。執行下列 SQL：

```
CREATE OR REPLACE TABLE `model_dev.vehicle_features_enhanced` AS
SELECT
  meta.auction_id,
  meta.item_name,
  meta.model_year,
  meta.make,
  meta.model,
  meta.mileage,
  meta.current_bid,
  meta.listing_url,
  meta.transmission_type,
  meta.description,
  meta.state,
  COALESCE(vision.body_style, 'Unknown') AS body_style,
  COALESCE(vision.condition, 50) AS condition,
  COALESCE(meta.color, vision.color, 'Unknown') AS color,
  COALESCE(vision.interior, 'Unknown') AS interior,
  COALESCE(scam.authenticity_score, 100) AS authenticity_score,
  vision.description_summary,
  prices.predicted_market_value
FROM `model_dev.vehicle_metadata` meta
LEFT JOIN `model_dev.vehicle_vision_features` vision
  ON meta.auction_id = vision.auction_id
LEFT JOIN `model_dev.vehicle_price_predictions` prices
  ON meta.auction_id = prices.auction_id
LEFT JOIN `model_dev.vehicle_authenticity_scores` scam
  ON meta.auction_id = scam.auction_id;
```

2. 接著，系統會結合四種不同的 AI 信號，計算出 0 到 100 分的**交易分數**。這項公式會平衡價值、品質和風險，找出最合適的房源：

- **價格分數 (40%)**：評估與公允市場價值相比的節省金額。
- **影像分數 (30%)**：先前相片分析的洞察資料。
- **真實性分數 (15%)**：詐騙風險評估。
- **狀況分數 (15%)**：透過 `AI.SCORE` 從賣家說明即時推斷。

執行下列查詢：

```

CREATE OR REPLACE TABLE `model_dev.marketplace_listings` AS
WITH score_elements AS (
  SELECT
    *,
    -- 1. SELLER DESCRIPTION SCORE (use AI.SCORE on seller description)
    AI.SCORE(
      FORMAT("Rate the vehicle condition (0-100) based ONLY on this text: '%s'",
description)
    ) AS description_score,

    -- 2. PRICE SCORE
    -- Higher impact for underpricing, lower impact for overpricing.
    CAST(LEAST(100.0, GREATEST(0.0,
      75.0 + (
        IF((predicted_market_value - current_bid) > 0,
          ((predicted_market_value - current_bid) / NULLIF(predicted_market_value, 0)) *
250.0,
          ((predicted_market_value - current_bid) / NULLIF(predicted_market_value, 0)) *
40.0
        )
      )
    )) AS INT64) AS price_score
  FROM `model_dev.vehicle_features_enhanced`
),
final_calcs AS (
  SELECT
    *,

    -- 3. Combine scores: Price (40%), Condition (30%), Description (15%), Authenticity (15%)
    ROUND(
      (
        (price_score * 0.40) +
        (CAST(condition AS INT64) * 0.30) +
        (COALESCE(description_score, 50) * 0.15) +
        (CAST(authenticity_score AS INT64) * 0.15)
      )
      -- Authenticity penalty for scores below 50.
      * (IF(CAST(authenticity_score AS INT64) < 50, 0.20, 1.05))
    ) AS raw_score
  FROM score_elements
)
SELECT
  * EXCEPT(raw_score),

  -- 4. Set floor values: low authenticity scores drop to 10; others floor at 35.
  CAST(GREATEST(
    (IF(CAST(authenticity_score AS INT64) < 50, 10, 35)),
    LEAST(100, raw_score)
  ) AS INT64) AS deal_score
FROM final_calcs;

```

為確保建議內容品質優良，查詢會套用兩個特定邏輯層：

- 1. **真實性闡道**：如果房源遭到標記為「高風險」(分數低於 50)，系統會自動將總交易分數調降 80%，避免宣傳可疑房源。
- 2. **「隱藏寶石」最佳化**：公式會使用分段邏輯，積極獎勵節省的金額，同時對加價較寬容，確保即使車輛狀況良好但價格過高，仍可獲得「一般」排名。

注意：結果會以表格形式具體化，避免在每次載入前端網頁時重新執行 LLM。

產生的資料表 `model_dev.marketplace_listings` 包含 `deal_score` 等欄位，以及 `price_score` 和 `authenticity_score`。

- 3. 如要自行查看交易分數，請執行下列查詢，或直接參閱下方的螢幕截圖：

```
SELECT item_name, model_year, authenticity_score, predicted_market_value, price_score, deal_score FROM `model_dev.marketplace_listings`
```

Row	item_name	model_year	authenticity_score	predicted_market...	price_score	deal_score
1	1996 Chevrolet Astro	1996	84	4219.23	42	57.0
2	2002 Ford Windstar LX Minivan	2002	85	3940.24	79	69.1
3	2005 Ford Expedition	2005	82	1952.76	78	66.3
4	2007 Ford Explorer	2007	67	6143.41	10	40.6
5	2008 Ford Escape	2018	86	9730.46	80	65.3

本節重點：您結合了預測價格、視覺化功能和真實性分數，以及賣家說明，為每個商品資訊計算出單一的特惠分數。

步驟 8 · 約 5 分鐘

8. 部署前端應用程式

現在可以啟動前端應用程式。這樣一來，您就能搜尋車輛資訊的庫存，並與剛建立的 AI 生成洞察資料互動，例如交易分數。

將 AI 分數匯出至前端

React 前端會依賴本機 JSON 酬載，快速載入初始網頁。如要為市集提供支援，請從 BigQuery 擷取最終的生成式交易分數，然後將這些分數重新注入 Next.js 專案。

1. 確認環境已準備就緒。如果 Cloud Shell 工作階段逾時，或是您前往其他資料夾，請執行下列指令返回專案根目錄，並還原環境變數：

```
cd ~/devrel-demos/data-analytics/cymbal-autos-multimodal && \  
export PROJECT_ID=$(gcloud config get-value project) && \  
export USER_BUCKET="cymbal-autos-${PROJECT_ID}"
```

2. 執行提供的 Python 指令碼，查詢最終的 BigQuery 檢視畫面，並將新的交易分數合併至應用程式的基礎資料儲存庫：

```
python3 scripts/setup/08_export_frontend_data.py
```

你會收到類似以下的確認訊息：

```
 Updated local file: app/src/data/cars.json
```

將應用程式部署至 Cloud Run

資料擴充作業完成後，您可以使用 Cloud Run 將 Next.js 前端應用程式部署至公開網際網路。這個網站提供現代化介面，包含交易評分、互動式圖片輪播，以及可即時查詢 BigQuery 的動態混合語意搜尋列。

1. 在 Cloud Shell 中，前往所複製存放區的 **app/** 目錄。這點非常重要，因為留在根目錄會導致建構失敗。

```
cd app
```

2. 使用 Cloud Run 將應用程式部署為無伺服器容器。這項指令會將 **PROJECT_ID** 做為環境變數傳遞，讓 Next.js API 知道要查詢哪個 BigQuery 專案：

```
gcloud run deploy cymbal-autos-frontend \  
  --source . \  
  --region us-west1 \  
  --allow-unauthenticated \  
  --min-instances 1 \  
  --set-env-vars PROJECT_ID=$PROJECT_ID \  
  --project $PROJECT_ID
```

傳遞 `--min-instances 1` 可防止冷啟動。這項功能可讓 Next.js API 路由保持運作，因此在查詢 BigQuery 時，語意搜尋列會立即回應。

3. 部署完成後，終端機會輸出安全的服務網址。內容如下所示：

```
Service URL: https://cymbal-autos-frontend-[YOUR-PROJECT-NUMBER].us-west1.run.app/
```

如要部署正式版，強烈建議[使用 Identity-Aware Proxy \(IAP\) 保護 Cloud Run 服務](#)，確保只有授權使用者可以存取應用程式。這項功能會新增以身分為依據的中央安全防護層，可驗證每個要求的使用者身分和環境，不必對應用程式程式碼進行複雜的變更。

9. 探索 Cymbal Autos 應用程式

前端容器已推送至 Cloud Run，現在可以測試應用程式了。

注意：由於交易分數是根據多模態模型發出的信號計算，且模擬車輛資訊資料具有一定程度的程序生成特性，因此您的具體交易分數和搜尋結果可能會與下方螢幕截圖略有不同。

1. 造訪網站：開啟 Cloud Run 傳回的安全服務網址。

 **Cymbal Autos**

[Inventory](#) [Financing](#) [How our AI works](#)

Find Your Hidden Gem

We analyze thousands of data points and photos using Google Cloud AI to find you the safest, best-priced vehicles on the market.

[Search Inventory](#)

Featured Deals



DEAL SCORE
86 (Excellent)

2016 Ford Explorer 4X4
78,289 mi • Montrose, CO
Specifications Make: FORD Model: EXPLORER 4X4
Model Year: 2016 Transmission Type: Automatic No ...
\$ **3,025**



DEAL SCORE
83 (Great)

2011 Ford Explorer
93,730 mi • Bismarck, ND
Overview Featured for sale is this 2011 Ford Explorer.
The vehicle is in usable condition. The vehicle is...
\$ **8,135**



DEAL SCORE
82 (Great)

2009 Ford Motor Co Escape Hybrid
63,896 mi • Lakewood, WA
Overview 2009 Ford Escape 4x2 Hybrid. The vehicle
is in repairable condition and is currently inoperative...
\$ **5,499**

2. 執行語意搜尋：嘗試搜尋抽象概念，例如「可靠的貨車，可拖曳且能越野」。Next.js 應用程式會將原始文字轉換為多模態向量嵌入，並對 BigQuery 觸發即時 `VECTOR_SEARCH`，將您的想法對應至車輛生態系統。

Find Your Hidden Gem

We analyze thousands of data points and photos using Google Cloud AI to find you the safest, best-priced vehicles on the market.

[🔍 Search Inventory](#)

注意：清單會依語意相似度排序。

3. 查看結果：BigQuery 會計算抽象概念與車輛特徵之間的確切數學距離，並傳回最接近的語意比對結果。

[🔍 Search Inventory](#)

Search Results



DEAL SCORE
⚠️ **40** (Poor)

2016 Chevrolet Silverado 2500

88,841 mi • Rock Springs, WY

Specifications Make: Chevrolet Model: Silverado 2500
Model Year: 2016 Transmission Type: Automatic No of...

\$ 17,799



DEAL SCORE
😊 **53** (Fair)

2018 Ford Motor Co F-350

152,555 mi • Rock Springs, WY

Specifications Make: Ford Motor Co Model: F350 Model
Year: 2018 Transmission Type: Automatic No of...

\$ 18,510

4. 深入瞭解詳細資料：按一下任一車輛，即可開啟完整刊登資料。

5. 檢查 AI 信號：捲動詳細資料，查看您稍早在實驗室中產生的原始機器學習分數：

-  公允市場價值：XGBoost 模型預測的基準價格。

- ✨ 外觀狀況：Gemini 模型擷取的實體損壞程度評估。
- 🔍 認證分數：認證向量指標可區分合法賣家和潛在詐騙者。

2016 Ford Explorer 4X4

78,289 miles • Montrose, CO

PRICE
\$3,025



Excellent Deal

86

Intelligent Deal Score (out of 100)

Under The Hood Breakdown

 Market Value (i) **100 / 100**

Price is \$5,486.96 below predicted fair market value.
Predicted Value: \$8,511.96

✨ Visual Condition (i) **72 / 100**

Based on multimodal analysis of listing photos.

🔍 Authenticity Score (i) **High Risk**

Authenticity rating based on semantic distance from known fraudulent listings (score: 45/100).

Contact Seller • \$3,025

10. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，您可以[刪除為本程式碼研究室建立的整個 Google Cloud 專案](#)，也可以執行下列自動終止指令碼。

1. 從 Cloud Shell 終端機返回包含目錄的根目錄：

```
cd ..
```

2. 執行下方的清除指令碼。這會清空 Google Cloud Storage bucket、捨棄 `model_dev` BigQuery 資料集、刪除 BigQuery 連線，以及刪除 Cloud Run 服務。

```
chmod +x scripts/cleanup/teardown.sh  
./scripts/cleanup/teardown.sh
```

11. 恭喜

恭喜！您已成功建立智慧車輛市集。您使用 BigQuery 將非結構化資料分析、預測模型和 AI 整合功能整合到單一工作區。

目前所學內容

- 如何使用 [ObjectRef](#) 將 BigQuery 連結至非結構化 Cloud Storage 圖片
- 如何使用 BigQuery 和 [AI.GENERATE](#) 和 [AI.CLASSIFY](#) 函式等 Gemini 模型，從相片中擷取車輛屬性
- 如何使用 [BigQuery ML](#) 預測車輛價格
- 如何嵌入車輛說明並執行 [VECTOR_SEARCH](#)，找出潛在的詐騙車輛資訊
- 如何使用 [AI.SCORE](#) 即時評估非結構化資料，並將結果納入全面的交易分數
- 如何匯出資料，並將 Next.js Marketplace 應用程式部署至 [Cloud Run](#)

後續步驟

- 瞭解 [BigQuery](#) 提供的全方位生成式 AI 函式
 - 進一步瞭解如何使用 [GoogleSQL](#) 建立預測模型
-